CustomScrollView

What is `CustomScrollView` in Flutter?

`CustomScrollView` is a scrollable widget in Flutter that allows you to create **scrolling effects with multiple types of children**—not just one scrollable list or column. It's more flexible than `ListView` or `SingleChildScrollView` because it supports multiple **slivers**—scrollable areas that can behave differently.

Why Use `CustomScrollView`?

You use `CustomScrollView` when you need **advanced scroll effects** or **a mix of scrollable content types**, such as:

- A header that collapses as you scroll (like `SliverAppBar`)
- A list followed by a grid
- Custom scroll animations or layout behavior
- Sticky headers or dynamic scroll behavior

Unlike `ListView`, which is a simple linear list, `CustomScrollView` lets you define more complex and optimized scrollable layouts.

Key Widgets (Slivers) Used in `CustomScrollView`

`CustomScrollView` works by composing different types of **slivers**. Here are the most important ones:

1. SliverAppBar

- A scrollable app bar that can expand, collapse, and stick at the top.
- Often used for flexible space with images or animations.

```
SliverAppBar(
  expandedHeight: 200.0,
  floating: false,
  pinned: true,
  flexibleSpace: FlexibleSpaceBar(
    title: Text("My App Bar"),
    background: Image.asset('assets/header.jpg', fit:
BoxFit.cover),
  ),
)
```

2. SliverList

- A sliver that displays linear items like a `ListView`.

```
SliverList(
  delegate: SliverChildBuilderDelegate(
    (BuildContext context, int index) {
      return ListTile(title: Text("Item #$index"));
    },
    childCount: 20,
  ),
```

```
)
```

3. SliverGrid

- A sliver that displays items in a grid (like `GridView`).

```
SliverGrid(
  gridDelegate:
SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2,
  ),
  delegate: SliverChildBuilderDelegate(
    (BuildContext context, int index) {
      return Container(
        color: Colors.blue,
        child: Center(child: Text('Grid
$index')),
      );
    },
    childCount: 10,
  ),
)
```

4. SliverToBoxAdapter

- Allows non-sliver widgets (e.g., a `Container`, `Image`, or `Text`) inside a `CustomScrollView`.

```
SliverToBoxAdapter(
  child: Container(
    height: 100,
    color: Colors.red,
    child: Center(child: Text("Hello from Box")),
  ),
)
```

5. SliverFillRemaining

- Fills the remaining space in the viewport. Useful for placing content that should stick to the bottom if there's not enough scrollable content.

```
SliverFillRemaining(
  hasScrollBody: false,
  child: Center(child: Text("This sticks to
bottom")),
)
```

Sample CustomScrollView Structure

```
CustomScrollView(
  slivers: <Widget>[
    SliverAppBar(
      expandedHeight: 200.0,
      pinned: true,
      flexibleSpace: FlexibleSpaceBar(title:
```

```
Text('Demo')),
    ),
    SliverToBoxAdapter(
      child: Padding(
        padding: EdgeInsets.all(8.0),
        child: Text("Intro Section"),
      ),
    ),
    SliverList(
      delegate: SliverChildBuilderDelegate(
        (context, index) => ListTile(title:
Text("Item $index")),
        childCount: 10,
      ),
    ),
    SliverGrid(
      delegate: SliverChildBuilderDelegate(
        (context, index) => Container(
          color: Colors.teal,
          child: Center(child: Text("Grid $index")),
        ),
        childCount: 6,
      ),
      gridDelegate:
SliverGridDelegateWithFixedCrossAxisCount(crossAxisCo
unt: 2),
    ),
  ],
```

)

Summary

Feature	`CustomScrollView` Advantage
Multiple scroll types	Combines list, grid, headers, and more
Performance	Efficient lazy loading with slivers
Flexibility	Allows sticky headers, collapsing toolbars, complex layouts
Reusability	You can compose and reuse slivers in different ways